

Resumo

Este handout apresenta o Document Object Model (DOM): a representação em árvore que o navegador constrói a partir de um documento HTML e que permite ao JavaScript acessar e modificar o conteúdo, a estrutura e o estilo de uma página em tempo real. São abordados em detalhe a estrutura da árvore DOM — nós, tipos de nós, relações hierárquicas e o CSSOM — e a interface JavaScript padrão para seleção, navegação, leitura e escrita de conteúdo, atributos, classes e estilos, além da criação, inserção e remoção dinâmica de elementos. O tratamento de eventos é discutido com profundidade, incluindo propagação, delegação e os eventos mais relevantes para o desenvolvimento web. Ao final, apresenta-se o jQuery como uma biblioteca que simplifica e padroniza essa interface, com comparação direta à API nativa moderna.

gian@uffs.edu.br

Versão: março de 2026

Keywords:

DOM; Document Object Model; árvore DOM; nós; CSSOM; Render Tree; querySelector; classList; addEventListener; propagação; delegação; jQuery; seletores

Como usar este material: Boxes verdes trazem dicas práticas. Boxes amarelos propõem exercícios. Boxes vermelhos alertam para armadilhas comuns. Boxes azuis apresentam contexto importante. Todo código deve ser digitado — não copiado — para consolidar o aprendizado. Abra o console do navegador (F12 → aba Console) e execute os exemplos em tempo real enquanto lê o handout.

1. O Que É o DOM

Quando o navegador recebe um arquivo HTML pela rede, ele não simplesmente exibe o texto recebido. Antes de qualquer renderização, ele executa uma etapa chamada de **parsing**: analisa o HTML caractere por caractere e constrói uma representação interna do documento em memória. O resultado é uma estrutura de dados em forma de árvore — o **DOM** (*Document Object Model*).

O DOM tem três papéis simultâneos e inseparáveis:

- **Modelo:** representa a estrutura do documento como uma hierarquia de objetos JavaScript, onde cada elemento HTML, cada nó de texto, cada atributo e cada comentário é representado por um objeto com propriedades e métodos.
- **Interface:** expõe uma API padronizada que permite ao JavaScript ler e modificar qualquer aspecto do documento — conteúdo, estrutura, estilo e atributos — em tempo real, sem recarregar a página.
- **Ponte:** conecta o HTML estático (estrutura) ao JavaScript dinâmico (comportamento). Qualquer mudança feita no DOM é imediatamente refletida na tela — o navegador redesenha apenas as partes afetadas.

DOM não é HTML, não é JavaScript e não é exclusivo do navegador O DOM é uma interface independente de linguagem, definida pela especificação WHATWG/W3C como um padrão web aberto. Ele pode, em teoria, ser implementado em qualquer linguagem — existem implementações em Python (lxml), Java e PHP. No contexto do navegador, JavaScript é a linguagem que acessa o DOM através do objeto global `document`, que representa o nó raiz da árvore e expõe todos os métodos de seleção e manipulação. O DOM também não é o HTML — o HTML é o texto que deu origem ao DOM, mas os dois podem divergir: o navegador corrige erros no HTML antes de construir o DOM, e o JavaScript pode modificar o DOM sem alterar o HTML original.

2. A Árvore DOM

2.1. Da marcação HTML à árvore de objetos

Para cada elemento HTML, atributo e trecho de texto, o navegador cria um **nó** (*node*) na árvore. Considere o seguinte documento:

```

<!DOCTYPE html>
<html lang="pt-BR">
  <head>
    <title>GEX613</title>
  </head>
  <body>
    <h1 id="titulo">Programação II</h1>
    <p class="intro">Disciplina do <strong>4º semestre</strong>.</p>
    <ul>
      <li>Node.js</li>
      <li>Express</li>
    </ul>
  </body>
</html>

```

Código 1. HTML de exemplo que será transformado em árvore DOM.

O navegador constrói a seguinte árvore de nós a partir deste HTML:

```

Document
+-- DOCTYPE: html
+-- html [lang="pt-BR"]
  +-- head
    | +-- #text "\n      "      <- no de texto (whitespace)
    | +-- title
    |   +-- #text "GEX613"
  +-- body
    +-- h1 [id="titulo"]
    |   +-- #text "Programacao II"
    +-- p [class="intro"]
    |   +-- #text "Disciplina do "
    |   +-- strong
    |       +-- #text "4 semestre"
    |       +-- #text "."
    +-- ul
        +-- li
        |   +-- #text "Node.js"
        +-- li
        |   +-- #text "Express"

```

Código 2. Representação em texto da árvore DOM gerada pelo HTML acima.

Há dois aspectos importantes a observar nessa árvore. Primeiro, o espaço em branco entre as tags (quebras de linha e indentação) gera **nós de texto** — são invisíveis na tela, mas estão presentes na árvore e podem afetar a navegação por `firstChild` e `nextSibling`. Segundo, os atributos como `id="titulo"` e `class="intro"` são parte do nó do elemento e acessíveis através de suas propriedades, não como nós filhos separados.

2.2. Relações hierárquicas entre nós

Cada nó da árvore conhece seus vizinhos. O vocabulário preciso dessas relações é importante para ler documentação e código:

- **Pai** (*parent node*): o nó imediatamente acima na hierarquia. O elemento `body` é pai de `h1`, `p` e `ul`. O `document` é pai de `html`.
- **Filho** (*child node*): um nó imediatamente abaixo de outro. `li` é filho de `ul`; `strong` é filho de `p`.
- **Irmão** (*sibling node*): dois nós com o mesmo pai. `h1`, `p` e `ul` são irmãos entre si.
- **Ancestral** (*ancestor*): qualquer nó acima na hierarquia, não apenas o pai imediato. `html` é ancestral de todos os outros nós. `body` é ancestral de `strong`.
- **Descendente** (*descendant*): qualquer nó abaixo na hierarquia. `strong` é descendente tanto de `p` quanto de `body` e de `html`.

2.3. Tipos de nós

Nem tudo na árvore DOM é um elemento HTML. A especificação do DOM define doze tipos de nós, cada um com um número inteiro identificador (`nodeType`). Os mais importantes no dia a dia são:

Na prática, a esmagadora maioria das operações envolve nós do tipo 1 (Element) e do tipo 3 (Text). Os métodos de seleção como `querySelector` retornam exclusivamente

Tipo	Nome	Constante	Exemplo
1	Elemento	ELEMENT_NODE	<p>, <div>, <h1>
2	Atributo	ATTRIBUTE_NODE	id="titulo" (raramente acessado diretamente)
3	Texto	TEXT_NODE	O texto "Programação II" dentro de <h1>
8	Comentário	COMMENT_NODE	<!-- comentário -->
9	Documento	DOCUMENT_NODE	O objeto document em si
10	Tipo de documento	DOCUMENT_TYPE_NODE	<!DOCTYPE html>
11	Fragmento	DOCUMENT_FRAGMENT_NODE	Um fragmento temporário em memória

Tabela 1. Principais tipos de nós no DOM.

nós do tipo 1. Conhecer os outros tipos é importante para entender o comportamento de `childNodes` e `firstChild`, que incluem nós de texto e comentários.

```
const h1 = document.querySelector('h1');

console.log(h1.nodeType); // 1 (ELEMENT_NODE)
console.log(h1.nodeName); // "H1" (sempre em maiúsculas para elementos)
console.log(h1.nodeValue); // null (elementos não têm valor direto)

// Nó de texto dentro do h1 (via firstChild, que inclui texto)
const textoH1 = h1.firstChild;
console.log(textoH1.nodeType); // 3 (TEXT_NODE)
console.log(textoH1.nodeName); // "#text"
console.log(textoH1.nodeValue); // "Programação II"

// O document em si
console.log(document.nodeType); // 9 (DOCUMENT_NODE)
console.log(document.nodeName); // "#document"
```

Código 3. Inspecionando o tipo e o nome de nós via JavaScript.

2.4. O CSSOM e a Render Tree

Paralelamente ao DOM, o navegador constrói um segundo modelo em árvore: o **CSSOM** (*CSS Object Model*). O CSSOM representa todas as regras CSS aplicadas ao documento, vindas de arquivos `.css` externos, de tags `<style>` internas e de atributos `style= inline`.

O navegador combina o DOM com o CSSOM para gerar a **Render Tree** (*árvore de renderização*). A Render Tree contém apenas os nós *visíveis* do documento, cada um com todos os seus estilos calculados (*computed styles*). Elementos com `display: none` são excluídos da Render Tree (mas permanecem no DOM); elementos com `visibility: hidden` permanecem na Render Tree (mas são desenhados transparentes).

É com base na Render Tree que o navegador realiza as duas últimas etapas: **layout** (cálculo das posições e dimensões) e **paint** (pintura dos pixels na tela). Qualquer modificação no DOM ou nos estilos pode disparar um novo ciclo de layout e paint — o que é chamado de *reflow* ou *repaint*, dependendo do que foi alterado.

```
const h1 = document.querySelector('h1');

// .style lê apenas estilos definidos INLINE no atributo style=""
// Se o estilo vem de um .css externo, .style retorna string vazia
console.log(h1.style.color); // "" (vazio, a menos que seja inline)
console.log(h1.style.fontSize); // "" (idem)

// getComputedStyle: retorna os estilos CALCULADOS pelo navegador
// Resultado de herança + cascata + valores padrão do navegador
const cs = window.getComputedStyle(h1);
console.log(cs.color); // "rgb(31, 78, 121)" valor real
console.log(cs.fontSize); // "32px" em pixels, mesmo que definido em rem
console.log(cs.display); // "block"
console.log(cs.fontFamily); // "Inter, Arial, sans-serif"

// getComputedStyle é somente leitura não é possível atribuir valores
```

Código 4. Acessando o CSSOM: estilo inline vs. estilo computado.

Reflow vs. Repaint **Reflow** (ou *layout*) acontece quando uma mudança afeta a geometria do layout — tamanho, posição, margem, padding. É a operação mais cara: o navegador precisa recalcular posições de todos os descendentes afetados. Disparado por: alterar `width`, `height`, `font-size`, adicionar ou remover elementos do DOM. **Repaint** acontece quando uma mudança afeta a aparência mas não a geometria — cor, sombra, visibilidade. É menos caro que o reflow, pois não precisa recalcular posições. Para manter a performance, evite fazer múltiplas modificações no DOM em sequência — agrupe-as (use `DocumentFragment` ou modifique o `innerHTML` uma única vez) para minimizar o número de reflows.

3. Selecionando Elementos

O ponto de entrada para qualquer operação no DOM é obter uma referência ao(s) elemento(s) que se deseja manipular. O objeto `document` oferece vários métodos de seleção; a API moderna é baseada em seletores CSS.

3.1. `querySelector` e `querySelectorAll`

Introduzidos com o Selectors API Level 1 (implementado em todos os navegadores modernos), `querySelector` e `querySelectorAll` aceitam qualquer seletor CSS válido como argumento — tornando toda a expressividade do CSS disponível para a seleção de elementos via JavaScript.

```
// Seletor de elemento retorna o primeiro h1 da página
const titulo = document.querySelector('h1');

// Seletor de classe retorna o primeiro .card encontrado
const primCard = document.querySelector('.card');

// Seletor de ID
const btnLogin = document.querySelector('#btn-login');

// Seletor de atributo
const emailInput = document.querySelector('input[type="email"]');

// Seletor composto (descendente)
const navLink = document.querySelector('nav a'); // 1o link dentro de nav

// Seletor de pseudo-classe
const primeiroLi = document.querySelector('ul li:first-child');

// Retorna null se nenhum elemento corresponder ao seletor
const inexistente = document.querySelector('.nao-existe'); // null
if (inexistente) { /* ... */ } // sempre verifique antes de usar

// Retorna NodeList estática com todos os elementos que casam
const todosCards = document.querySelectorAll('.card');
const todosLinks = document.querySelectorAll('a');
const inputsForm = document.querySelectorAll('form input, form select');

console.log(todosCards.length); // quantidade de elementos encontrados

// NodeList suporta forEach diretamente
todosCards.forEach(card => {
  console.log(card.textContent);
});

// Para usar map, filter, reduce: converter para Array primeiro
const textos = Array.from(todosLinks).map(a => a.textContent);

// Spread também converte NodeList em Array
const arr = [...todosCards];

// querySelectorAll retorna NodeList ESTÁTICA:
```

Código 5. `querySelector`: retorna o primeiro elemento encontrado.

Código 6. `querySelectorAll`: retorna todos os elementos encontrados.

```
// não reflete alterações futuras no DOM (ao contrário de HTMLCollection)
```

💡 **Restrinja a busca ao contexto correto.** `querySelector` e `querySelectorAll` funcionam em qualquer `Element`, não apenas no `document`. Ao chamar em um elemento específico, a busca é restrita aos seus descendentes — o que é mais eficiente e evita selecionar elementos fora do contexto desejado:

```
const form = document.querySelector('form');
const inputs = form.querySelectorAll('input'); // só inputs DENTRO do form

const nav = document.querySelector('nav');
const links = nav.querySelectorAll('a'); // só links dentro da nav
```

3.2. Métodos de seleção clássicos

Anteriores ao Selectors API, esses métodos ainda são amplamente encontrados em código legado e em situações específicas onde são ligeiramente mais eficientes:

```
// getElementById: o mais rápido para busca por ID
// Retorna Element ou null
const btn = document.getElementById('btn-salvar');

// getElementsByName: retorna HTMLCollection VIVA
// (atualizada automaticamente quando o DOM muda)
const cards = document.getElementsByName('card');

// getElementsByTagName: idem, por nome de tag
const paragrafos = document.getElementsByTagName('p');

// getElementsByName: por atributo name (usado em formulários)
const radios = document.getElementsByName('genero');

// CUIDADO com HTMLCollection viva: se você adicionar/remover
// elementos com a classe 'card' durante a iteração, o length
// muda e você pode pular elementos ou iterar em loop infinito!
// Solução: converter para Array antes de iterar
Array.from(cards).forEach(c => c.classList.add('visivel'));
```

Código 7. Métodos de seleção herdados e suas peculiaridades.

⚠ **HTMLCollection viva vs. NodeList estática.** `getElementsByName` e `getElementsByTagName` retornam uma **HTMLCollection viva**: se você adicionar um elemento com a classe buscada ao DOM durante a iteração, ele aparece imediatamente na coleção, podendo causar loops infinitos ou iterações incorretas. `querySelectorAll` retorna uma **NodeList estática**: é uma cópia do estado do DOM no momento da chamada e não muda durante a iteração. Para código novo, prefira `querySelectorAll` e converta para `Array` quando precisar de métodos funcionais.

4. Navegando na Árvore DOM

Uma vez que você tem uma referência a um nó, pode navegar até seus vizinhos usando as propriedades de travessia. Há duas famílias de propriedades: as que consideram todos os tipos de nó (incluindo texto e comentários) e as que filtram apenas nós do tipo `Element`. Na prática, quase sempre se usa a família *Element* para evitar a complexidade dos nós de texto gerados por whitespace.

Navegação por nós (inclui texto e comentários)

```
const ul = document.querySelector('ul');

// Todos os filhos, incluindo nós de texto
console.log(ul.childNodes); // NodeList [#text, li, #text, li, #text]
console.log(ul.firstChild); // #text (whitespace antes do 1o <li>)
console.log(ul.lastChild); // #text (whitespace após o último <li>)
```

Código 8. Propriedades de travessia que incluem todos os tipos de nó.

```
console.log(ul.childNodes.length); // 5 (2 li + 3 nós de texto whitespace)

// Pai e irmãos (qualquer tipo de nó)
console.log(ul.parentNode); // body
console.log(ul.previousSibling); // #text (whitespace antes do <ul>)
console.log(ul.nextSibling); // #text (whitespace após o <ul>)
```

Navegação por elementos (ignora texto e comentários)

```
const ul = document.querySelector('ul');

// Versões "Element" ignoram nós de texto e comentários
console.log(ul.children); // HTMLCollection [li, li]
console.log(ul.firstChild); // primeiro <li>
console.log(ul.lastElementChild); // último <li>
console.log(ul.childElementCount); // 2

const li1 = ul.firstChild;
console.log(li1.parentElement); // ul
console.log(li1.nextElementSibling); // segundo <li>
console.log(li1.previousElementSibling); // null (é o primeiro)
```

Código 9. Propriedades de travessia que retornam apenas Elements.

Buscando ancestrais: closest()

O método `closest(seletor)` percorre a árvore **de baixo para cima**, começando no próprio elemento, e retorna o primeiro ancestral que corresponde ao seletor CSS fornecido. Retorna `null` se nenhum ancestral corresponder. É especialmente útil em handlers de eventos, onde o elemento clicado pode ser um filho profundo de um container de interesse:

```
const li = document.querySelector('ul li:last-child');

// Sobe na hierarquia até encontrar um <ul>
const lista = li.closest('ul'); // o <ul> pai
const main = li.closest('main'); // o <main> ancestral, ou null

// Uso clássico: detectar cliques em qualquer parte de um card
document.addEventListener('click', function(e) {
  const card = e.target.closest('.card');
  if (!card) return; // clicou fora de um card

  const id = card.dataset.id;
  console.log('Card clicado, id:', id);
});
// Funciona mesmo que o clique ocorra em um <span> dentro do .card

// contains(): verifica se um elemento contém outro
const nav = document.querySelector('nav');
const link = document.querySelector('a');
console.log(nav.contains(link)); // true se link é descendente de nav
```

Código 10. `closest()` para busca eficiente de ancestrais.

5. Lendo e Alterando Conteúdo

5.1. `textContent`, `innerHTML` e `innerText`

Essas três propriedades permitem ler e escrever o conteúdo de elementos, mas com comportamentos distintos que é fundamental compreender:

```
const p = document.querySelector('p.intro');
// HTML atual: <p class="intro">Disciplina do <strong>4º semestre</strong></p>

// textContent: texto puro de todos os descendentes, sem nenhuma tag
console.log(p.textContent);
// "Disciplina do 4º semestre."

// innerHTML: conteúdo HTML como string, incluindo todas as tags internas
console.log(p.innerHTML);
// "Disciplina do <strong>4º semestre</strong>."
```

Código 11. Diferença entre `textContent`, `innerHTML` e `innerText`.

```
// innerText: similar a textContent, mas considera a renderização CSS.
// Elementos com display:none são ignorados. Aplica quebras de linha
// para <br> e blocos. É mais lento pois força um reflow.
console.log(p.innerText);
// "Disciplina do 4º semestre."
// (aqui igual ao textContent, mas difere em casos com CSS)

const div = document.querySelector('#resultado');

// textContent: interpreta o valor como texto puro SEGURO com dados externos
// Qualquer < ou > no valor é exibido como texto, não como HTML
div.textContent = 'Bem-vindo, <strong>visitante</strong>!';
// Exibe literalmente: Bem-vindo, <strong>visitante</strong>!

// innerHTML: renderiza o valor como HTML USE APENAS com conteúdo confiável
div.innerHTML = 'Bem-vindo, <strong>visitante</strong>!';
// Renderiza: Bem-vindo, visitante! (com "visitante" em negrito)

// Limpar o conteúdo de um elemento
div.innerHTML = ''; // ou div.textContent = '';

// NUNCA faça isso com dados do usuário:
// const nome = req.query.nome; // poderia ser: <img onerror="roubarDados()">
// div.innerHTML = 'Olá ' + nome; // executa JavaScript malicioso!
```

Código 12. Escrevendo conteúdo: segurança e diferenças.

⚠ innerHTML com dados externos é uma vulnerabilidade XSS. Atribuir innerHTML com qualquer valor que venha do usuário — campos de formulário, parâmetros de URL, dados de APIs externas — pode executar JavaScript malicioso no navegador de quem visita a página. Este tipo de ataque é chamado de XSS (*Cross-Site Scripting*) e é uma das vulnerabilidades mais exploradas na web. Regra: use textContent quando o valor for texto puro. Se precisar inserir HTML dinâmico, use a biblioteca DOMPurify: `div.innerHTML = DOMPurify.sanitize(htmlExterno)`.

5.2. Valores de elementos de formulário

Elementos de formulário expõem propriedades específicas para leitura e escrita de seus valores — que não são acessíveis via textContent ou innerHTML:

```
const email    = document.querySelector('input[name="email"]');
const checkbox = document.querySelector('input[type="checkbox"]');
const select   = document.querySelector('select#curso');
const textarea = document.querySelector('textarea');
const radio    = document.querySelector('input[type="radio"]:checked');

// Lendo valores
console.log(email.value);      // texto digitado no campo
console.log(checkbox.checked);   // true se marcado, false se não
console.log(select.value);     // value da opção selecionada
console.log(textarea.value);   // texto da área de texto
console.log(radio?.value);     // valor do radio selecionado (ou undefined)

// Escrevendo (útil para preencher ou limpar programaticamente)
email.value      = 'usuario@uffs.edu.br';
checkbox.checked    = true;
select.value     = 'CC'; // seleciona a opção com value="CC"

// Utilitários de foco e seleção
email.focus();    // move o foco do teclado para o campo
email.blur();     // remove o foco
email.select();   // seleciona todo o texto do campo
email.setSelectionRange(0, 5); // seleciona apenas os 5 primeiros chars
```

Código 13. Propriedades de valor em elementos de formulário.

6. Atributos HTML e Classes CSS

6.1. Manipulando atributos HTML

Atributos HTML podem ser lidos e modificados de duas formas: através da API `getAttribute/setAttribute`, que trabalha com atributos HTML exatamente como escritos no markup, ou através de propriedades refletidas diretamente no objeto do elemento (`element.href`, `element.src`, etc.), que são sincronizadas bidirecionalmente com os atributos mas podem ter valores ligeiramente diferentes (como URLs absolutas em vez de relativas).

```
const img = document.querySelector('img');

// Lendo atributos  getAttribute sempre retorna string ou null
console.log(img.getAttribute('src'));    // 'foto.jpg'
console.log(img.getAttribute('alt'));    // 'Foto do campus'
console.log(img.getAttribute('width'));  // '800' (SEMPRE string!)
console.log(img.getAttribute('hidden')); // null se não existe

// Verificando existência
console.log(img.hasAttribute('hidden')); // false

// Escrevendo e criando
img.setAttribute('alt', 'Vista aérea do Campus Chapecó');
img.setAttribute('loading', 'lazy');    // adiciona atributo novo

// Removendo
img.removeAttribute('width');

// Propriedades refletidas (mais direto para atributos padrão HTML)
const link = document.querySelector('a');
console.log(link.href);    // URL ABSOLUTA: "https://uffs.edu.br" (!)
console.log(link.getAttribute('href')); // valor original: "/sobre"
link.target = '_blank';    // adiciona target="_blank"
link.rel = 'noopener';    // adiciona rel="noopener"
```

Código 14. API `getAttribute`, `setAttribute`, `removeAttribute`.

Atributos `data-*` e o `dataset`

O HTML5 introduziu os atributos `data-*` como mecanismo oficial para armazenar dados privados no HTML que não têm semântica predefinida. O JavaScript os expõe através da propriedade `dataset`, que mapeia automaticamente os nomes de atributos (kebab-case) para propriedades (camelCase):

```
// HTML: <article class="card" data-id="42"
//         data-tipo-usuario="professor" data-ultimo-acesso="2026-03">

const card = document.querySelector('.card');

// dataset espelha os atributos data-* como propriedades JavaScript
// data-id          -> dataset.id
// data-tipo-usuario -> dataset.tipoUsuario (kebab camelCase)
// data-ultimo-acesso -> dataset.ultimoAcesso
console.log(card.dataset.id);    // "42"
console.log(card.dataset.tipoUsuario); // "professor"
console.log(card.dataset.ultimoAcesso); // "2026-03"

// Criando: adiciona data-status="ativo" ao elemento
card.dataset.status = 'ativo';

// Removendo
delete card.dataset.ultimoAcesso;

// Iterando todos os data-* attributes
for (const [chave, valor] of Object.entries(card.dataset)) {
  console.log(chave, '=', valor);
}
```

Código 15. Atributos `data-*` e a propriedade `dataset`.

💡 Use `data-*` para associar dados a elementos sem classes extras. Atributos `data-*` são o lugar correto para armazenar informações que o JavaScript precisará ler — como IDs de registros, estados ou configurações. Isso é preferível a usar classes CSS para carregar dados (que polui o sistema de classes) ou a armazenar dados em variáveis globais separadas do elemento. `data-id`, `data-acao` e `data-estado` são padrões muito comuns em aplicações web modernas.

6.2. Manipulando classes CSS com `classList`

A propriedade `className` expõe as classes de um elemento como uma única string. Manipulá-la diretamente é frágil: `element.className = 'nova-classe'` apaga todas as classes existentes. A API `classList` resolve isso oferecendo métodos granulares:

```
const btn = document.querySelector('.btn');

// Adicionar, remover e verificar individualmente
btn.classList.add('ativo');           // adiciona sem duplicar
btn.classList.remove('desabilitado'); // remove se existir, ignora se não
btn.classList.toggle('expandido');    // add se não tem, remove se tem

// toggle com segundo argumento: força o estado
btn.classList.toggle('visivel', true); // sempre adiciona
btn.classList.toggle('visivel', false); // sempre remove

// Verificar
console.log(btn.classList.contains('ativo')); // true
console.log(btn.classList.length);           // nº de classes
console.log([...btn.classList]);              // array de strings

// Substituir uma classe por outra
btn.classList.replace('btn-azul', 'btn-verde'); // retorna true/false

// Múltiplas classes de uma vez
btn.classList.add('visivel', 'animado', 'destaque');
btn.classList.remove('oculto', 'desabilitado');

// Iterar
btn.classList.forEach(cls => console.log(cls));

// Evite: apaga TODAS as classes existentes
// btn.className = 'btn ativo'; // substitui tudo!
```

Código 16. `classList`: a API correta para manipular classes.

7. Manipulando Estilos

7.1. A propriedade `style`

A propriedade `style` de um elemento representa seus estilos *inline* — equivalente ao atributo `style=` no HTML. As propriedades CSS são acessadas em `camelCase` (sem hífen):

```
const div = document.querySelector('.card');

// Escrevendo: camelCase para propriedades com hífen
div.style.backgroundColor = '#fff3cd'; // background-color
div.style.fontSize        = '1.25rem'; // font-size
div.style.marginTop       = '2rem';    // margin-top
div.style.borderRadius    = '8px';

// Ocultando e mostrando elementos
div.style.display = 'none'; // oculta (equivalente a display:none no CSS)
div.style.display = '';     // remove o estilo inline, restaura o CSS externo
div.style.display = 'block'; // mostra explicitamente como bloco

// Lendo: retorna string vazia se o estilo não for inline
console.log(div.style.backgroundColor); // '#fff3cd' (se foi definido inline)
```

Código 17. Lendo e escrevendo estilos inline com `.style`.

```

console.log(div.style.color);           // '' (se veio do .css externo)

// cssText: define múltiplos estilos de uma vez (substitui todos os inline)
div.style.cssText = 'color: #333; font-size: 1rem; padding: 1rem;';

// Para ler os estilos REAIS (resultado de herança, cascata e padrões):
const cs = window.getComputedStyle(div);

console.log(cs.color);                 // 'rgb(51, 51, 51)' sempre em rgb/rgba
console.log(cs.fontSize);              // '20px' sempre em px
console.log(cs.display);              // 'block'
console.log(cs.fontFamily);           // 'Inter, Arial, sans-serif'
console.log(cs.marginTop);            // '32px' computado em pixels

// getComputedStyle é somente leitura não é possível atribuir valores

// Pseudo-elementos também podem ser inspecionados
const antes = window.getComputedStyle(div, '::before');
console.log(antes.content); // '' ou 'none'

```

Código 18. `getComputedStyle`: lendo o estilo real do elemento.

💡 **Prefira classes em vez de estilos inline.** Alterar `element.style` aplica CSS inline, que tem a maior especificidade possível e é difícil de sobrescrever. A prática recomendada é definir todas as variações de estilo como classes no arquivo `.css` e usar `classList.toggle()` para ativá-las. Isso mantém os estilos no lugar correto e o JavaScript apenas controla o estado — separação de responsabilidades aplicada.

```

/* No CSS: */
.card.expandido { height: 300px; padding: 2rem; }

```

```

// No JavaScript:
card.classList.toggle('expandido'); // em vez de card.style.height = '300px'

```

8. Criando e Removendo Elementos

8.1. Criando elementos e inserindo no DOM

A inserção de elementos no DOM ocorre em duas etapas independentes: primeiro cria-se o elemento em memória com `document.createElement()`, configura-se seu conteúdo e atributos, e depois ele é inserido na árvore com um dos métodos de inserção.

```

// 1. Criar
const li = document.createElement('li');

// 2. Configurar conteúdo e atributos
li.textContent = 'Sequelize';
li.classList.add('item-topico');
li.dataset.modulo = '4';
li.setAttribute('title', 'ORM para Node.js');

// 3. Inserir na árvore
const ul = document.querySelector('ul');
ul.appendChild(li); // como ÚLTIMO filho retorna o nó inserido

// Criando elemento com estrutura interna via innerHTML
// (somente com conteúdo confiável nunca com dados do usuário)
const card = document.createElement('article');
card.className = 'card';
card.innerHTML = `
  <h3 class="card__titulo">Handout 03 APIs REST</h3>
  <p class="card__descricao">Express, rotas e middlewares.</p>
  <a class="card__link" href="handout03.pdf">Baixar</a>
`;
document.querySelector('main').appendChild(card);

```

Código 19. Criando, configurando e inserindo elementos.

Métodos modernos de inserção

```
const ul = document.querySelector('ul');
const novoLi = document.createElement('li');
novoLi.textContent = 'pg-promise';

// prepend: insere como PRIMEIRO filho (aceita strings e Elements)
ul.prepend(novoLi);
ul.prepend('Texto no início'); // insere nó de texto

// append: insere como ÚLTIMO filho (mais moderno que appendChild)
ul.append(novoLi);
ul.append('Texto no final', document.createElement('li')); // múltiplos

// before / after: insere como irmão, FORA do elemento pai
const li1 = ul.firstElementChild;
li1.before(novoLi); // novoLi fica ANTES de li1 (mesmo nível)
li1.after(novoLi); // novoLi fica DEPOIS de li1

// insertAdjacentHTML: insere HTML sem descartar o conteúdo existente
// Posições: 'beforebegin' | 'afterbegin' | 'beforeend' | 'afterend'
ul.insertAdjacentHTML('beforeend', '<li class="novo">JWT</li>');
ul.insertAdjacentHTML('afterbegin', '<li class="primeiro">Reset global</li>');

// insertAdjacentElement e insertAdjacentText: versões para Element e texto
ul.insertAdjacentElement('beforeend', novoLi);
```

Código 20. prepend, append, before, after e insertAdjacentHTML.

Para pensar: Com base nos métodos before() e after(), escreva uma função moverParaCima(elemento) que troca um item de lista com seu irmão anterior. E uma moverParaBaixo(elemento) que troca com o próximo. Use previousElementSibling e nextElementSibling para verificar se o movimento é possível.

8.2. Clonando e removendo elementos

```
const li1 = document.querySelector('ul li');

// cloneNode(true): clona o elemento E todos os seus descendentes
// cloneNode(false): clona apenas o elemento, sem filhos
const copia = li1.cloneNode(true);
copia.textContent = 'Express (cópia)';
li1.parentElement.appendChild(copia);

// Remover um elemento da árvore (forma moderna ES2015)
const ultiLi = document.querySelector('ul li:last-child');
ultiLi.remove(); // remove a si mesmo da árvore

// Forma clássica (remove um filho a partir do pai)
const ul = document.querySelector('ul');
ul.removeChild(ul.lastElementChild);

// Substituir um elemento por outro (método moderno ES2019)
const novoTitulo = document.createElement('h2');
novoTitulo.textContent = 'Tópicos do Semestre';
document.querySelector('h1').replaceWith(novoTitulo);

// Forma clássica
ul.replaceChild(novoTitulo, ul.firstElementChild);
```

Código 21. cloneNode, remove, replaceWith e replaceChild.

8.3. DocumentFragment: inserções em lote eficientes

Cada vez que você insere um elemento no DOM, o navegador pode disparar um *reflow*. Se você estiver inserindo muitos elementos em sequência, isso resulta em múltiplos reflows caros. O DocumentFragment resolve isso: é um contêiner leve que existe apenas em memória, não na árvore do documento, e que não dispara reflows enquanto você o monta. Ao inserir o fragmento no DOM, ocorre apenas um único reflow.

```
const topicos = [
  { titulo: 'Node.js', url: '/node' },
  { titulo: 'Express', url: '/express' },
  { titulo: 'Sequelize', url: '/sequelize' },
  { titulo: 'JWT', url: '/jwt' },
];

const ul = document.querySelector('ul');

// Ineficiente: um reflow por appendChild (4 reflows)
topicos.forEach(t => {
  const li = document.createElement('li');
  li.textContent = t.titulo;
  ul.appendChild(li);
});

// Eficiente: um único reflow ao inserir o fragmento
const frag = document.createDocumentFragment();
topicos.forEach(t => {
  const li = document.createElement('li');
  const a = document.createElement('a');
  a.href = t.url;
  a.textContent = t.titulo;
  li.appendChild(a);
  frag.appendChild(li); // manipula em memória, sem tocar no DOM
});
ul.appendChild(frag); // um único reflow aqui
```

Código 22. DocumentFragment para inserção eficiente.

9. Eventos

Eventos são o mecanismo pelo qual o navegador notifica o JavaScript de que algo aconteceu — um clique, uma tecla, o carregamento da página, o envio de um formulário. O JavaScript registra **ouvintes de evento** (*event listeners*) que são chamados automaticamente quando o evento ocorre.

9.1. addEventListener e removeEventListener

O método `addEventListener(tipo, handler)` registra uma função para ser chamada quando o evento especificado ocorrer no elemento. É a forma moderna e recomendada de trabalhar com eventos — ao contrário dos atributos HTML inline como `onclick="..."`, que misturam estrutura e comportamento.

```
const btn = document.querySelector('#btn-salvar');

// Forma básica
btn.addEventListener('click', function(event) {
  console.log('Clicado!');
  console.log(event.target); // elemento que originou o evento
});

// Com arrow function (mais conciso)
btn.addEventListener('mouseover', (e) => {
  e.target.style.opacity = '0.8';
});

// Para remover, é necessário guardar referência à função
function aoClicar(e) {
  console.log('clique registrado');
}
btn.addEventListener('click', aoClicar);
btn.removeEventListener('click', aoClicar); // remove o mesmo handler

// Opção once: executa uma única vez e remove automaticamente
btn.addEventListener('click', () => console.log('apenas uma vez'), {
  once: true
});
```

Código 23. Registrando ouvintes de evento.

9.2. O objeto Event

Todo handler de evento recebe um objeto `Event` como primeiro argumento. Esse objeto contém informações sobre o que ocorreu e métodos para controlar o comportamento padrão do evento:

```
document.querySelector('form').addEventListener('submit', (e) => {
  // Prevenir o comportamento padrão (recarregar a página ao submeter)
  e.preventDefault();

  // Informações sobre o evento
  console.log(e.type);           // 'submit'
  console.log(e.target);         // o elemento <form>
  console.log(e.currentTarget);  // o elemento onde o listener está registrado
  console.log(e.timeStamp);      // número de milissegundos desde o carregamento

  // Dados de formulário via FormData
  const dados = new FormData(e.target);
  console.log(dados.get('email')); // valor do input name="email"
});

// Eventos de teclado
document.addEventListener('keydown', (e) => {
  console.log(e.key);           // 'Enter', 'Escape', 'a', 'ArrowUp'
  console.log(e.code);          // 'Enter', 'KeyA', 'ArrowUp' (posição física)
  console.log(e.ctrlKey);       // true se Ctrl está pressionado
  console.log(e.shiftKey);      // true se Shift está pressionado
  console.log(e.altKey);        // true se Alt está pressionado

  if (e.key === 'Escape') fecharModal();
  if (e.ctrlKey && e.key === 's') { e.preventDefault(); salvar(); }
});
```

Código 24. Propriedades e métodos do objeto `Event`.

Eventos de mouse

```
const area = document.querySelector('.area-interativa');

area.addEventListener('mousemove', (e) => {
  // Posição relativa ao viewport
  console.log(e.clientX, e.clientY);

  // Posição relativa ao documento (inclui scroll)
  console.log(e.pageX, e.pageY);

  // Posição relativa ao elemento alvo
  console.log(e.offsetX, e.offsetY);
});

area.addEventListener('click', (e) => {
  console.log(e.button); // 0 = esquerdo, 1 = meio, 2 = direito
});

// mouseenter/leave NÃO fazem bubbling (ao contrário de mouseover/out)
area.addEventListener('mouseenter', () => area.classList.add('hover'));
area.addEventListener('mouseleave', () => area.classList.remove('hover'));
```

Código 25. Eventos de mouse e suas propriedades.

9.3. Propagação de eventos: Bubbling e Capturing

Quando um evento ocorre em um elemento, ele não fica restrito a esse elemento. O evento percorre a árvore DOM em três fases:

1. **Capturing** (captura): o evento desce do `window` até o elemento alvo.
2. **Target**: o evento está no elemento alvo.
3. **Bubbling** (borbulhamento): o evento sobe do elemento alvo de volta até o `window`.

Por padrão, `addEventListener` registra o handler na fase de bubbling (subida). Para registrar na fase de capturing, passe `true` como terceiro argumento.

```
// Dado: <div class="pai"> <button class="filho">Clique</button> </div>

document.querySelector('.pai').addEventListener('click', () => {
  console.log('pai clicado'); // também executa quando o filho é clicado!
});

document.querySelector('.filho').addEventListener('click', (e) => {
  console.log('filho clicado');
  e.stopPropagation(); // impede o evento de subir para o pai
});

// stopImmediatePropagation: impede outros listeners NO MESMO ELEMENTO
// (além de parar o bubbling)
btn.addEventListener('click', (e) => {
  e.stopImmediatePropagation();
  console.log('este executa');
});
btn.addEventListener('click', () => {
  console.log('este NÃO executa'); // bloqueado pelo anterior
});
```

Código 26. Bubbling, stopPropagation e stopImmediatePropagation.

9.4. Delegação de eventos

A delegação de eventos é um padrão que aproveita o bubbling para registrar um único listener no elemento pai em vez de um em cada filho. Isso é mais eficiente para listas longas e é a única solução para elementos adicionados dinamicamente ao DOM após o registro do listener:

```
const ul = document.querySelector('#lista-tarefas');

// Ineficiente: um listener por item (e não cobre itens futuros)
document.querySelectorAll('#lista-tarefas li').forEach(li => {
  li.addEventListener('click', () => li.classList.toggle('concluida'));
});

// Delegação: um único listener no pai
ul.addEventListener('click', (e) => {
  // closest encontra o <li> ancestral do elemento clicado
  const li = e.target.closest('li');
  if (!li || !ul.contains(li)) return; // clicou fora de um <li>

  // Verificar qual botão foi clicado pelo data-acao
  if (e.target.matches('[data-acao="concluir"]')) {
    const id = li.dataset.id;
    marcarConcluida(id);
  } else if (e.target.matches('[data-acao="remover"]')) {
    li.remove();
  } else {
    li.classList.toggle('selecionada');
  }
});

// Itens adicionados DEPOIS do listener ainda funcionam!
const novoItem = document.createElement('li');
novoItem.dataset.id = '99';
novoItem.innerHTML =
  'Nova tarefa <button data-acao="remover">X</button>';
ul.appendChild(novoItem); // automaticamente coberto pelo listener
```

Código 27. Delegação: um listener no pai cobre todos os filhos.

DOMContentLoaded vs. load

```
// DOMContentLoaded: disparado quando o HTML foi completamente parseado
// e o DOM está construído CSS e imagens podem ainda estar carregando
document.addEventListener('DOMContentLoaded', () => {
  // seguro para selecionar qualquer elemento do DOM
  const btn = document.querySelector('#btn-principal');
  btn.addEventListener('click', iniciar);
});
```

Código 28. DOMContentLoaded, load e a preferência pelo defer.

Categoria	Eventos e descrição
Mouse	click (1 clique), dblclick (2 cliques), mouseenter/mouseleave (sem bubbling), mouseover/mouseout (com bubbling), contextmenu (botão direito)
Teclado	keydown (tecla pressionada), keyup (tecla solta). keypress é obsoleto.
Formulário	submit (envio), change (valor alterado e foco perdido), input (cada caractere), focus/blur (foco ganho/perdido), reset
Documento	DOMContentLoaded (HTML parseado), load (tudo carregado), beforeunload (antes de fechar), scroll, resize
Toque	touchstart, touchmove, touchend (dispositivos móveis)
Drag & Drop	dragstart, drag, dragenter, dragover, drop, dragend

Tabela 2. Principais eventos DOM por categoria.

```
// load: disparado quando TODO o conteúdo foi carregado:
// HTML + CSS + imagens + fontes + iframes
window.addEventListener('load', () => {
  // use quando precisar das dimensões reais de imagens ou posições
  const img = document.querySelector('img.hero');
  console.log(img.naturalWidth, img.naturalHeight);
});

// Com <script defer> no <head>, o script só executa após o HTML ser
// parseado equivalente a DOMContentLoaded. Nesse caso, não é
// necessário envolver todo o código em DOMContentLoaded.
```

Para pensar: Implemente uma lista de tarefas completa: (1) formulário com input de texto e botão de adicionar; (2) ao submeter o formulário, crie um novo com o texto digitado, um botão de “concluir” e um botão de “remover”; (3) use delegação de eventos no para lidar com os botões; (4) mantenha um contador de tarefas pendentes atualizado. Implemente com DocumentFragment para a inserção inicial de itens.

10. jQuery

jQuery é uma biblioteca JavaScript criada por John Resig em 2006 com dois objetivos principais: simplificar a manipulação do DOM e abstrair as inconsistências entre os navegadores — um problema grave na época, quando Internet Explorer, Firefox e Safari tinham APIs DOM muito diferentes entre si.

Embora os navegadores modernos sejam muito mais padronizados (tornando o segundo objetivo obsoleto) e a API DOM nativa seja suficiente para a maioria dos casos, o jQuery permanece amplamente presente em projetos legados, em plataformas como WordPress e em plugins de UI. Compreender jQuery é importante tanto para trabalhar com código existente quanto para apreciar por que a API nativa evoluiu na direção que tomou.

10.1. Incluindo jQuery

```
<!-- jQuery deve ser carregado ANTES dos scripts que o utilizam -->
<!-- Posicione antes do </body> ou com defer no <head> -->
<script
  src="https://code.jquery.com/jquery-3.7.1.min.js"
  integrity="sha256-JqT3SQfawRcv/BIHPTkBs00EvtFfmqPF/1YI/Cxo="
  crossorigin="anonymous">
</script>
<script src="meu-script.js"></script>
```

Código 29. Incluindo jQuery via CDN com Subresource Integrity.

10.2. A função \$ e os objetos jQuery

O coração do jQuery é a função `$()` (também acessível como `jQuery()`). Ela recebe um seletor CSS e retorna um **objeto jQuery** — um objeto que encapsula um ou mais elementos DOM e expõe uma API fluente. O objeto jQuery não é um Element nativo: para acessar o elemento subjacente, use `[0]` ou `.get(0)`.

```
// JavaScript nativo           // jQuery equivalente
document.querySelector('h1')   // $('h1')
document.querySelectorAll('.card') // $('.card')
document.getElementById('btn') // $('#btn')

// jQuery também aceita seletores CSS avançados
$('nav a');                    // todos <a> dentro de <nav>
$('input:not([disabled])');    // inputs não desabilitados
$('li:nth-child(2)');          // segundo <li> de cada pai

// O objeto jQuery não é um Element use [0] para obter o Element nativo
const $btn = $('#btn-salvar');
console.log($btn[0]);          // Element nativo
console.log($btn.get(0));       // idem, mais explícito
console.log($btn.length);       // 1 (ou 0 se não encontrado)

// Aguardar o DOM (equivalente a DOMContentLoaded)
$(document).ready(function() { /* código aqui */ });
$(function() { /* forma abreviada */ });
```

Código 30. Seleção com jQuery vs. API nativa.

10.3. Lendo e alterando conteúdo e atributos

Uma característica marcante do jQuery é que os métodos de leitura e escrita são o mesmo método — o que é chamado de padrão *getter/setter* implícito: sem argumento, lê; com argumento, escreve.

```
// text(): getter/setter de textContent
const texto = $('h1').text();           // lê o textContent
$('h1').text('Novo título');            // define

// html(): getter/setter de innerHTML
const markup = $('p').html();            // lê o innerHTML
$('#resultado').html('<strong>OK</strong>');

// val(): getter/setter de value (inputs, select, textarea)
const email = $('input[name="email"]').val(); // lê
$('input[name="email"]').val('x@y.com');      // define

// attr(): getter/setter de atributos HTML
const src = $('img').attr('src');         // lê
$('img').attr('alt', 'Nova descrição');    // define um
$('a').attr({ href: '/nova', target: '_blank' }); // define múltiplos

// removeAttr()
$('input').removeAttr('disabled');

// prop(): para propriedades DOM (checked, disabled, selected...)
$('input[type="checkbox"]').prop('checked', true);
console.log($('input[type="checkbox"]').prop('checked')); // true
```

Código 31. Conteúdo, HTML, valor e atributos no estilo getter/setter.

⚠ attr() vs. prop() no jQuery. No jQuery, `attr()` acessa atributos HTML (`getAttribute` / `setAttribute`) e `prop()` acessa propriedades DOM. Para `checked`, `selected`, `disabled`, `value`, `href` e outros atributos que têm propriedades DOM correspondentes, use sempre `prop()` — o resultado é mais previsível. `attr('checked')` retorna `"checked"` ou `undefined` (o atributo HTML); `prop('checked')` retorna `true` ou `false` (o estado atual do checkbox).

10.4. Classes, estilos e visibilidade

```
// Classes
$('.btn').addClass('ativo');
$('.btn').removeClass('desabilitado');
$('.btn').toggleClass('expandido');
$('.btn').toggleClass('ativo', condicao); // true = add, false = remove
console.log($('.btn').hasClass('ativo')); // true/false

// css(): getter/setter de estilos
console.log($('.card').css('font-size')); // lê estilo computado
$('.card').css('background-color', '#fff3cd'); // define um
$('.card').css({ // define múltiplos
  'font-size': '1.1rem',
  'border': '1px solid #2e75b6',
  'border-radius': '8px'
});

// Visibilidade: hide/show/toggle são equivalentes a display:none/block
$('.modal').hide(); // display: none
$('.modal').show(); // display: block
$('.menu').toggle(); // alterna

// Com duração: animação de fade
$('.modal').hide(300); // fade out em 300ms
$('.modal').show('fast'); // equivale a 200ms
$('.modal').fadeIn(400); // fade in
$('.modal').fadeOut(400); // fade out
$('.menu').slideToggle(200); // desliza para abrir/fechar
```

Código 32. Classes, estilos e visibilidade com jQuery.

10.5. Criando, inserindo e removendo elementos

```
// Criar elemento passando HTML como string para $()
const $li = $('<li>').text('Prisma').addClass('item');
const $card = $('<article class="card"><h3>Título</h3></article>');

// Inserir como filho
$('.ul').append($li); // como último filho
$('.ul').prepend('<li>Reset</li>'); // como primeiro filho

// Inserir como irmão
$('.ul li:first').before($li); // antes do primeiro li
$('.ul li:last').after($li); // depois do último li

// Equivalentes invertidos (inserir o container ao redor do objeto)
$li.appendTo('ul'); // equivale a $('.ul').append($li)
$li.prependTo('ul');
$li.insertBefore('ul li:last');
$li.insertAfter('ul li:first');

// Remover
$('.card').remove(); // remove e destrói dados jQuery associados
$('.card').detach(); // remove mas preserva dados (para reinserir)

// Substituir
$('h1').replaceWith('<h2>Novo Título</h2>');

// Esvaziar: remove filhos, mantém o elemento
$('#lista').empty();

// Clonar
const $clone = $('li:first').clone(true); // true = clona também os listeners
$clone.text('Cópia'); $('.ul').append($clone);
```

Código 33. Manipulação da estrutura DOM com jQuery.

10.6. Eventos com jQuery

```
// on(): registra ouvinte (equivalente a addEventListener)
$('#btn-salvar').on('click', function(e) {
    e.preventDefault();
    // $(this) = o elemento que recebeu o evento, como objeto jQuery
    console.log('Texto do botão:', $(this).text());
});

// Múltiplos eventos com a mesma função
$('input').on('focus blur', function() {
    $(this).toggleClass('ativo');
});

// Namespace de evento: facilita remover handlers específicos
$('#btn').on('click.meuPlugin', handler);
$('#btn').off('click.meuPlugin'); // remove apenas esse handler

// Delegação: 2o argumento é o seletor do filho
$('ul').on('click', 'li', function() {
    $(this).toggleClass('selecionado');
});
// Equivalente a: ul.addEventListener('click', e => {
//   if (e.target.matches('li')) { ... } });

// off(): remove todos os listeners de um tipo
$('#btn').off('click');

// Atalhos para eventos comuns (são apenas on() abreviados)
$('#btn').click(handler);
$('form').submit(function(e) { e.preventDefault(); });
$('input').change(function() { console.log($(this).val()); });
$('input').on('input', function() { /* a cada caractere */ });
```

Código 34. Registrando e removendo eventos com jQuery.

10.7. Encadeamento (method chaining)

Como a maioria dos métodos jQuery retorna o próprio objeto jQuery, é possível encadear múltiplas operações em uma única expressão fluente. Isso reduz a repetição e torna o código mais legível quando aplicado com moderação:

```
// Sem encadeamento
const $el = $('#resultado');
$el.addClass('destaque');
$el.css('color', '#1f4e79');
$el.text('Operação concluída');
$el.fadeIn(300);

// Com encadeamento (mais compacto)
$('#resultado')
    .addClass('destaque')
    .css('color', '#1f4e79')
    .text('Operação concluída')
    .fadeIn(300);

// end(): volta para a seleção anterior no encadeamento
$('ul')
    .find('li:even')           // seleciona itens pares
    .addClass('par')
    .css('background', '#f0f4f8')
    .end()                   // volta para $('ul')
    .find('li:odd')
    .addClass('impar');

// Filtragem durante o encadeamento
$('.card')
    .filter('[data-tipo="destaque"]') // mantém apenas os destacados
    .addClass('borda-azul')
    .find('h3')                     // busca h3 dentro dos filtrados
```

Código 35. Encadeamento de métodos jQuery.

```
.css('font-size', '1.5rem');
```

10.8. jQuery vs. JavaScript moderno: comparação direta

Operação	jQuery	JS Nativo (ES6+)	
Selecionar (1)	<code>\$('.card')</code>	<code>querySelector('.card')</code>	
Selecionar (N)	<code>\$('.card')</code>	<code>querySelectorAll('.card')</code>	
Aguardar DOM	<code>\$(function){}</code>	<code>DOMContentLoaded</code>	
Texto	<code>\$el.text('x')</code>	<code>el.textContent = 'x'</code>	
HTML	<code>\$el.html('x')</code>	<code>el.innerHTML = 'x'</code>	
Valor input	<code>\$el.val()</code>	<code>el.value</code>	
Adicionar classe	<code>\$el.addClass('a')</code>	<code>el.classList.add('a')</code>	
Toggle classe	<code>\$el.toggleClass('a')</code>	<code>el.classList.toggle('a')</code>	
Estilo	<code>\$el.css('color', 'r')</code>	<code>el.style.color = 'r'</code>	
Atributo (ler)	<code>\$el.attr('href')</code>	<code>el.getAttribute('href')</code>	
Evento	<code>\$el.on('click', fn)</code>	<code>el.addEventListener('click', fn)</code>	
Delegação	<code>\$ul.on('click', 'li', fn)</code>	<code>ul.addEventListener</code> <code>closest</code>	+
Criar elemento	<code>\$('#').text('x')</code>	<code>createElement</code> <code>textContent</code>	+
Inserir final	<code>\$el.append(filho)</code>	<code>el.append(filho)</code>	
Remover	<code>\$el.remove()</code>	<code>el.remove()</code>	
AJAX	<code>\$.ajax({})</code>	<code>fetch(url)</code>	

Tabela 3. Equivalência entre jQuery e a API DOM nativa moderna.

Quando ainda faz sentido usar jQuery em 2025 Para novos projetos, a API DOM nativa (ES6+) cobre praticamente todos os casos de uso do jQuery sem nenhuma dependência externa. As razões válidas para usar jQuery hoje são: (1) manutenção de projetos legados onde ele já está presente; (2) plugins de terceiros que dependem de jQuery (ex.: certas bibliotecas de UI e sliders); (3) WordPress, onde jQuery é carregado por padrão no tema/admin. Em novos projetos, prefira a API nativa ou frameworks modernos como Vue.js, React ou Alpine.js — que oferecem reatividade real, estado gerenciável e componentes reutilizáveis, além de ecossistemas ativos.

11. Para Pensar e Implementar

Exercício 1 — Inspeccionando a árvore DOM: Abra qualquer página web no Chrome (F12). Na aba **Elements**, expanda a árvore HTML e observe os nós. No console, execute: `document.body.childNodes` e note os nós de texto (whitespace). Compare com `document.body.children` que mostra apenas Elements. Execute `document.querySelector('p').parentElement` para subir um nível, e `document.querySelectorAll('a').length` para contar links. Identifique um `TEXT_NODE` com `firstChild.nodeType`.

Exercício 2 — Seleção e leitura: Crie uma página HTML com um `<h1>`, dois parágrafos com classes diferentes, uma lista com três itens e um formulário com dois inputs. No console (sem modificar o HTML): (1) selecione o `<h1>` e leia seu `textContent`; (2) selecione todos os `<li>` e imprima cada um com `forEach`; (3) leia o valor atual de cada input com `.value`; (4) use `getComputedStyle` para ver o `font-size` real do `<h1>`.

Exercício 3 — Manipulação dinâmica: Construa uma lista de tarefas: (1) crie um `<ul id="tarefas">` vazio e um formulário com input + botão; (2) ao submeter, crie um `<li>` com o texto digitado e um botão de remover via `createElement`; (3) use `DocumentFragment` para inserção quando houver múltiplos itens iniciais; (4) ao clicar em remover, use `li.remove()`; (5) mantenha um `<span id="contador">` com o número de tarefas atuais.

Exercício 4 — Eventos e delegação: Refatore o exercício anterior para usar delegação: um único `addEventListener('click')` no `<ul>` em vez de um por botão de remover. Use `e.target.closest('[data-acao="remover"]')` para detectar o clique correto. Adicione também a possibilidade de marcar uma tarefa como concluída (toggle de classe `.concluida`) clicando no texto. Verifique que a delegação funciona para itens adicionados depois do listener.

Exercício 5 — Reescrita com jQuery: Reimplemente os exercícios 3 e 4 usando jQuery: seleção com `$()`, criação com `$('<li>')`, inserção com `.append()`, remoção com `.remove()`, classes com `.toggleClass()` e delegação com `$('#tarefas').on('click', '[data-acao="remover"]', fn)`. Compare o número de linhas e a clareza com a versão em JavaScript puro. Qual abordagem você preferiria em um projeto novo? E em um projeto que já usa jQuery?